

**TẬP ĐOÀN CÔNG NGHIỆP - VIỆN THÔNG QUÂN ĐỘI
TỔNG CÔNG TY CÔNG NGHIỆP CÔNG NGHỆ CAO VIETTEL
ĐƠN VỊ VHT**

Chương trình Viettel Digital Talent



BÁO CÁO NGHIÊN CỨU

**Nghiên cứu, phát triển công cụ xử lý dữ liệu
3D Gaussian Splatting phục vụ hệ thống
Thực tế tăng cường (AR)**

Đề tài:	Công cụ xử lý và chuyển đổi dữ liệu 3D Gaussian Splatting thành dữ liệu hình học phục vụ AR
Người thực hiện:	Nguyễn Quý Đức
Mentor hướng dẫn:	Hoàng Kim Đức (VHT)
Email liên hệ:	nguyenquyduc2.00@gmail.com

Hà Nội, tháng 7 năm 2026

Tóm tắt đề tài

3D Gaussian Splatting (3DGS) cho phép tái tạo và hiển thị cảnh 3D với chất lượng trực quan cao, tốc độ thời gian thực và kích thước dữ liệu phù hợp cho các ứng dụng tương tác. Tuy nhiên, dữ liệu 3DGS chủ yếu mô tả trường bức xạ bằng tập Gaussian có vị trí, hướng, tỷ lệ, độ mờ và màu sắc; bản thân biểu diễn này không cung cấp bề mặt hình học kín, collision mesh hay navigation mesh. Trong bối cảnh Thực tế tăng cường (AR), khoảng trống này làm cho vật thể ảo khó nhận biết vật cản, khó bị che khuất đúng bởi môi trường thật và khó di chuyển theo địa hình có ý nghĩa hình học.

Đề tài nghiên cứu và phát triển một công cụ xử lý tự động nhằm chuyển đổi dữ liệu 3DGS đầu vào ở định dạng `.ply` hoặc `.splat` thành bộ artifact phục vụ WebAR và desktop AR. Hệ thống bao gồm nhân xử lý Rust, CLI ở mức proof of concept và ứng dụng desktop Tauri/React là giao diện cuối cùng. Pipeline hiện tại đọc nguồn 3DGS, bake căn chỉnh tỷ lệ và trục lên, áp dụng các bộ lọc dữ liệu, voxel hóa bằng CPU hoặc GPU, trích xuất mesh, hỗ trợ adapter SuGaR/Poisson cho tái tạo bề mặt ngoài, sinh occlusion mesh, tùy chọn navigation mesh bằng rerecast, và đóng gói `scene.sog`, `occlusion.glb`, `manifest.json`, `webar.zip` cùng các artifact phụ trợ.

Báo cáo trình bày cơ sở lý thuyết, yêu cầu chức năng, thiết kế hệ thống, phương pháp xử lý, triển khai ứng dụng desktop, cách đánh giá và các hạn chế còn tồn tại. Phần thực nghiệm không đưa số liệu benchmark chưa được chốt; thay vào đó mô tả bộ chỉ số, lệnh benchmark và phạm vi kiểm thử cần duy trì để bảo đảm báo cáo không đánh tráo giữa kết quả đã có và kết quả dự kiến.

Từ khóa: 3D Gaussian Splatting; AR; WebAR; voxelization; GPU; occlusion mesh; navigation mesh; SuGaR; Poisson Surface Reconstruction; Tauri; Rust.

Mục lục

1	Mở đầu	1
1.1	Bối cảnh	1
1.2	Vấn đề nghiên cứu	1
1.3	Mục tiêu	2
1.4	Phạm vi	2
1.5	Đóng góp chính	2
1.6	Cấu trúc báo cáo	3
2	Cơ sở lý thuyết	3
2.1	3D Gaussian Splatting	3
2.2	Khoảng trống hình học của 3DGS trong AR	4
2.3	Geometry proxy cho AR	4
2.4	Yêu cầu của WebAR	5
2.5	Voxelization và mesh extraction	5
2.6	Reconstruction liên quan: SuGaR và Poisson	5
2.7	Navigation mesh và Recast	6
2.8	Nền tảng WebAR và desktop	6
3	Phân tích yêu cầu	7
3.1	Từ khoảng trống 3DGS đến yêu cầu hệ thống	7
3.2	Nhóm yêu cầu theo luận điểm nghiên cứu	7
3.3	Geometry awareness	8
3.4	Occlusion readiness	8
3.5	Navigation readiness	8
3.6	Scale và coordinate consistency	8
3.7	WebAR deliverability và hiệu năng	9
3.8	Tính tái lập và khả năng kiểm chứng	9
3.9	Phạm vi yêu cầu	9
4	Thiết kế hệ thống	10
4.1	Quan điểm thiết kế theo dataflow	10
4.2	Input boundary	10
4.3	Calibration Recipe boundary	11
4.4	Processing Config boundary	11
4.5	Reconstruction boundary	11
4.6	Export boundary	12
4.7	Các bất biến của pipeline	12
4.8	Lý do thiết kế phù hợp với đề tài	13

5	Phương pháp xử lý dữ liệu	13
5.1	Chuẩn hóa bảng dữ liệu Splat	13
5.2	Alignment bake	13
5.3	Scene transform trong desktop app	14
5.4	Lọc dữ liệu	14
5.5	Voxelization CPU/GPU	14
5.6	Fill và carve	15
5.7	Mesh extraction	15
5.8	Adapter SuGaR/Poisson	15
5.9	Navigation mesh bằng rerecast	16
5.10	Export và đóng gói	16
6	Triển khai ứng dụng desktop	16
6.1	Vai trò của desktop app	16
6.2	Workflow giao diện	17
6.3	Quản lý calibration state	17
6.4	Editor transform	18
6.5	Tauri command bridge	18
6.6	Preview và tương tác 3D	18
6.7	Hiển thị kết quả	19
6.8	Ý nghĩa triển khai	19
7	Đánh giá và kiểm thử	19
7.1	Nguyên tắc đánh giá	19
7.2	Kiểm thử Rust core	19
7.3	Kiểm thử GUI	20
7.4	Benchmark và metric	20
7.5	Chỉ số hình học	21
7.6	Đánh giá artifact	21
7.7	Đánh giá trực quan	21
7.8	Kế hoạch cập nhật số liệu	21
8	Hạn chế và hướng phát triển	22
8.1	Hạn chế hiện tại	22
8.2	Rủi ro kỹ thuật	23
8.3	Hướng phát triển ngắn hạn	23
8.4	Hướng phát triển dài hạn	23
8.5	Định hướng đánh giá tiếp theo	24
9	Kết luận	24

1. Mở đầu

1.1. Bối cảnh

3D Gaussian Splatting (3DGS) là một hướng biểu diễn cảnh 3D hiện đại, trong đó cảnh được mô tả bằng tập các Gaussian không gian ba chiều thay vì mesh tam giác truyền thống. Mỗi Gaussian thường mang vị trí, tỷ lệ theo các trục chính, hướng, độ mờ và hệ số màu. Khi được rasterize bằng pipeline phù hợp, tập Gaussian này có thể tạo ảnh chất lượng cao với tốc độ thời gian thực, đặc biệt hữu ích cho visual reconstruction, digital twin, tham quan ảo và các ứng dụng cần xem lại không gian đã quét.

Trong AR, yêu cầu không chỉ dừng ở hiển thị. Vật thể ảo phải biết phần nào của cảnh thật là vật cản, đâu là mặt đi lại, đâu là vùng cần che khuất và đâu là tỷ lệ thế giới thực. Một mô hình 3DGS thuần túy có thể nhìn rất thật nhưng lại không có bề mặt hình học rõ ràng để engine AR kiểm tra va chạm hoặc dựng occlusion. Nếu đặt một vật thể ảo vào cảnh, vật thể đó có thể xuyên qua tường, không đứng vững trên sàn, hoặc không bị đồ vật thật che đúng cách. Vấn đề cốt lõi của đề tài vì vậy là biến dữ liệu 3DGS từ biểu diễn phục vụ nhìn thấy thành dữ liệu hình học phục vụ tương tác.

Đề tài được thực hiện trong bối cảnh đơn vị VHT, thuộc Tập đoàn Công nghiệp - Viễn thông Quân đội, trong chương trình Viettel Digital Talent. Mục tiêu không phải xây dựng một renderer 3DGS hoàn chỉnh từ đầu, mà là phát triển công cụ xử lý sau tái tạo: đọc dữ liệu đầu vào, chuẩn hóa theo hệ tọa độ và tỷ lệ thật, lọc dữ liệu, tái tạo bề mặt hình học, trích xuất artifact phục vụ AR, và đóng gói đầu ra theo hướng dễ tích hợp với WebAR hoặc ứng dụng desktop.

1.2. Vấn đề nghiên cứu

Dữ liệu 3DGS có hai đặc điểm làm cho việc dùng trực tiếp trong AR trở nên khó khăn. Thứ nhất, dữ liệu là tập primitive xác suất, không phải manifold surface. Một Gaussian có thể đóng góp màu và độ mờ lên ảnh render nhưng không đồng nghĩa với một điểm trên bề mặt cứng. Thứ hai, dữ liệu tái tạo thường phụ thuộc vào hệ tọa độ của quá trình scan hoặc training; đơn vị đo, hướng trục lên và gốc tọa độ không mặc nhiên khớp với thế giới thật. Nếu bỏ qua hai đặc điểm này, hệ thống AR sẽ có hình ảnh đẹp nhưng tương tác kém tin cậy.

Từ đó, đề tài tập trung vào ba câu hỏi chính:

1. Làm thế nào để chuyển dữ liệu 3DGS đầu vào thành lưới hình học đủ ổn định cho occlusion và navigation mà vẫn giữ kích thước artifact phù hợp?
2. Làm thế nào để thao tác căn chỉnh tỷ lệ, hướng trục và transform cảnh đủ đơn giản trong GUI nhưng vẫn sinh recipe rõ ràng cho backend?
3. Làm thế nào để thiết kế pipeline có thể mở rộng giữa reconstruction nội bộ bằng voxel và reconstruction ngoài bằng SuGaR hoặc Poisson adapter?

Các câu hỏi trên được giải quyết bằng cách tách hệ thống thành nhân xử lý Rust, CLI proof of concept và ứng dụng desktop Tauri/React. Nhân Rust giữ trách nhiệm đọc, chuẩn hóa, lọc, voxel hóa, mesh extraction, navmesh và export. CLI dùng để chạy tự động, kiểm thử và benchmark. Ứng dụng desktop là giao diện cuối cùng để người dùng thao tác nguồn dữ liệu, preview 3DGS, đặt scale endpoints, chọn up axis, chỉnh rotation XYZ và position XYZ, chạy bake và lưu gói WebAR.

1.3. Mục tiêu

Mục tiêu tổng quát của đề tài là xây dựng công cụ xử lý dữ liệu 3DGS phục vụ hệ thống AR. Công cụ cần nhận đầu vào .ply hoặc .splat, sinh ra dữ liệu hình học có cấu trúc và đóng gói được thành đầu ra chuẩn. Mục tiêu này được cụ thể hóa thành các mục tiêu thành phần:

- Đọc và chuẩn hóa dữ liệu 3DGS từ các nguồn phổ biến, bảo toàn các thuộc tính cần thiết cho render và export.
- Cung cấp cơ chế căn chỉnh thực tế gồm khoảng cách scale, up axis và scene transform bằng rotation XYZ, position XYZ.
- Lọc dữ liệu lỗi hoặc nhiễu qua filter NaN, opacity, box, sphere, cluster và floater contribution.
- Tái tạo hình học bằng voxelization CPU/GPU, mesh extraction và tùy chọn adapter SuGaR/Poisson.
- Sinh *occlusion mesh* dưới dạng `occlusion.glb`; sinh *navigation mesh* tùy chọn cho profile phù hợp.
- Đóng gói đầu ra gồm `manifest.json`, `scene.sog`, `occlusion.glb`, `webar.zip` và các file navmesh nếu được bật.
- Duy trì cấu trúc kiểm thử đủ để bảo vệ các hợp đồng dữ liệu giữa GUI, Tauri command và Rust core.

1.4. Phạm vi

Phạm vi đề tài là công cụ xử lý và chuyển đổi dữ liệu, không phải toàn bộ hệ thống AR production. Ứng dụng desktop là sản phẩm cuối cùng của giao diện người dùng trong repository hiện tại; CLI đóng vai trò proof of concept, regression harness và công cụ benchmark. Hệ thống chưa đặt mục tiêu tự động nhận diện ngữ nghĩa như tường, sàn, bàn, ghế hay vật thể động. Các quyết định hình học vẫn dựa vào dữ liệu 3DGS, profile bake và một số tương tác người dùng tối thiểu.

Với floor calibration, codebase hiện tại không còn workflow chọn nhiều điểm sàn. Recipe căn chỉnh dùng `upAxis`, `floorNormal`, hai `scalePoints`, `scaleDistanceMeters` và `origin`. Trong GUI, `floorNormal` được suy ra trực tiếp từ up axis đã chọn, nghĩa là đây là cơ chế chọn hướng trục lên chứ chưa phải thuật toán tự động fit mặt phẳng sàn từ điểm. Vì vậy báo cáo mô tả đúng hiện trạng này, tránh diễn đạt rằng hệ thống đã có floor detection tự động.

1.5. Đóng góp chính

Đề tài có bốn đóng góp chính. Đóng góp thứ nhất là thiết kế pipeline chuyển 3DGS sang artifact hình học theo hướng tách biệt giữa render data và AR geometry. Đóng góp thứ hai là triển khai nhân xử lý Rust có thể chạy qua CLI hoặc Tauri command, bao gồm đọc nguồn, alignment bake, filter, voxelization, mesh extraction, navmesh và export. Đóng góp thứ ba là giao diện desktop cho phép thao tác trực quan với scene, đặt scale endpoints, chọn up axis, chỉnh rotation/position và chạy bake. Đóng góp thứ tư là hợp đồng artifact rõ ràng để WebAR runtime đọc được dữ liệu scene, occlusion mesh và metadata.

Dữ liệu 3DGS	.ply/.splat, Gaussian attributes, preview scene
Căn chỉnh thực tế	scale endpoints, up axis, rotation XYZ, position XYZ
Tái tạo hình học	CPU/GPU voxel, SuGaR/Poisson adapter, rerecast navmesh
WebAR bundle	scene.sog, occlusion.glb, manifest.json, webar.zip

Hình 1: Luồng mục tiêu của công cụ xử lý 3DGS phục vụ AR.

1.6. Cấu trúc báo cáo

Báo cáo gồm chín phần. Phần 1 trình bày bối cảnh, vấn đề, mục tiêu và phạm vi. Phần 2 tổng hợp cơ sở lý thuyết liên quan đến 3DGS, voxelization, Marching Cubes, SuGaR, Poisson, occlusion và navigation mesh. Phần 3 phân tích yêu cầu chức năng và phi chức năng. Phần 4 mô tả thiết kế hệ thống. Phần 5 trình bày phương pháp xử lý dữ liệu. Phần 6 mô tả triển khai ứng dụng desktop. Phần 7 trình bày kiểm thử và phương pháp đánh giá. Phần 8 nêu hạn chế và hướng phát triển. Phần 9 kết luận.

2. Cơ sở lý thuyết

2.1. 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) là một phương pháp biểu diễn và render cảnh 3D dựa trên tập các Gaussian trong không gian ba chiều [1]. Khác với mesh tam giác, 3DGS không mô tả cảnh bằng các đỉnh, cạnh và mặt có topology rõ ràng. Khác với point cloud truyền thống, mỗi phần tử của 3DGS không chỉ là một điểm rời rạc mà là một primitive có kích thước, hướng, opacity và màu. Khác với nhiều cách biểu diễn neural rendering như NeRF, 3DGS đưa cảnh về tập primitive tường minh hơn, nhờ đó có thể render thời gian thực hiệu quả hơn trong nhiều trường hợp.

Một Gaussian 3D thường được mô tả bởi tâm $\mu \in \mathbb{R}^3$, ma trận hiệp phương sai Σ , opacity α và thông tin màu. Về trục giác, μ xác định vị trí, Σ xác định hình dạng ellipsoid, còn α mô tả mức đóng góp của Gaussian vào ảnh render. Một hàm Gaussian có thể viết dưới dạng:

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

Trong triển khai thực tế, Σ thường không được lưu trực tiếp dưới dạng ma trận đầy đủ, mà được tham số hóa qua scale và rotation để thuận tiện tối ưu và render. Khi render, Gaussian 3D được chiếu lên không gian ảnh thành splat 2D. Các splat được tổng hợp theo opacity để tạo màu cuối cùng. Một dạng alpha compositing thường gặp có thể biểu diễn khái quát như sau:

$$C = \sum_i T_i \alpha_i c_i, \quad T_i = \prod_{j < i} (1 - \alpha_j)$$

Trong đó c_i là màu của splat thứ i , α_i là opacity sau khi chiếu và T_i là độ truyền qua tích lũy từ các splat phía trước. Công thức này cho thấy bản chất của 3DGS là phục vụ hình ảnh: primitive nào đóng góp vào màu ảnh, với độ mờ bao nhiêu, và theo thứ tự nào. Nó không trực tiếp trả lời câu hỏi: đâu là mặt cứng, đâu là vật cản, đâu là vùng có thể đi lại.

2.2. Khoảng trống hình học của 3DGS trong AR

Điểm mạnh lớn nhất của 3DGS cũng chính là nguồn gốc của khó khăn khi đưa vào AR. 3DGS có thể tái tạo hình ảnh rất thuyết phục, nhưng biểu diễn này không đảm bảo có bề mặt hình học kín, normal ổn định hoặc topology nhất quán. Một vùng tường trong ảnh có thể được tái tạo bằng nhiều Gaussian mềm chồng lên nhau; một cạnh bàn có thể là tập Gaussian kéo dài; một khoảng trống nhỏ có thể bị lấp bởi opacity tích lũy. Những dữ kiện này đủ tốt cho render, nhưng chưa đủ tốt cho suy luận hình học.

Trong AR, hệ thống cần nhiều thông tin hơn việc "nhìn giống thật". Nếu đặt một vật thể ảo vào phòng, vật thể đó phải bị tường thật che khuất khi đứng sau tường. Nếu thả một vật thể ảo xuống sàn, nó không được xuyên qua sàn hoặc đứng lơ lửng. Nếu có một agent ảo di chuyển trong cảnh, nó cần biết vùng nào có thể đi, vùng nào bị chặn, độ dốc nào còn chấp nhận được. 3DGS thuần túy không cung cấp các câu trả lời này dưới dạng dữ liệu mà engine AR có thể dùng trực tiếp.

Có thể hình dung ba lỗi điển hình nếu dùng 3DGS như một asset render thuần trong AR:

- **Occlusion sai:** vật thể ảo vẫn hiện ra dù đáng lẽ bị tường, bàn hoặc vật cản trong cảnh thật che khuất.
- **Collision sai:** vật thể ảo xuyên qua sàn, tường hoặc đồ vật vì runtime không có collision surface.
- **Navigation sai:** agent ảo không tìm được đường, hoặc tìm đường qua vùng không thể đi vì không có navigation mesh.

Vì vậy, bài toán của đề tài không phải là render 3DGS đẹp hơn. Bài toán là tạo thêm một lớp hình học từ dữ liệu 3DGS để phục vụ tương tác AR. Lớp hình học này không cần tái tạo mọi chi tiết thị giác, nhưng phải đủ ổn định, đúng tỷ lệ và đủ nhẹ để dùng trong runtime.

2.3. Geometry proxy cho AR

Trong đồ họa thời gian thực, geometry proxy là một mô hình hình học đơn giản hơn mô hình hiển thị nhưng giữ những đặc điểm cần thiết cho tính toán. Với AR, hai proxy quan trọng là occlusion mesh và navigation mesh. Occlusion mesh đại diện cho vật cản dùng để che khuất vật thể ảo hoặc kiểm tra va chạm. Navigation mesh mô tả vùng có thể di chuyển, thường được rút gọn từ bề mặt cảnh theo các tham số như chiều cao agent, bán kính agent, độ dốc tối đa và khả năng leo bậc.

Occlusion mesh và navigation mesh có mục tiêu khác nhau. Occlusion mesh cần bám vào hình dạng vật cản đủ tốt để tạo cảm giác vật thể ảo nằm trong cảnh thật. Nó không nhất thiết phải có texture hoặc topology đẹp, nhưng phải có vị trí đúng và số tam giác vừa phải. Navigation mesh lại cần mô tả vùng đi được một cách ổn định. Một mesh đẹp về mặt hình ảnh chưa chắc tạo navmesh tốt; ví dụ mặt bàn có thể là mặt phẳng, nhưng về ngữ nghĩa không nhất thiết là vùng đi lại. Trong phạm vi đề tài, hệ thống tập trung vào geometry processing, chưa đặt mục

tiêu semantic understanding để tự hiểu đây là bàn, ghế, tường hay cửa.

Điều này dẫn đến một nguyên tắc thiết kế: dữ liệu render và dữ liệu tương tác nên được tách ra. `scene.sog` có thể giữ 3DGS để hiển thị cảnh. `occlusion.glb` và `navmesh` tùy chọn cung cấp geometry proxy cho AR. Tách hai lớp này giúp hệ thống tận dụng được ưu điểm thị giác của 3DGS mà vẫn có dữ liệu hình học cho runtime.

2.4. Yêu cầu của WebAR

WebAR đặt thêm các ràng buộc thực dụng. Trình duyệt và thiết bị di động thường có giới hạn về bộ nhớ, thời gian tải, GPU và network. Một mesh rất chi tiết có thể tốt cho kiểm tra hình học nhưng lại không phù hợp nếu làm `webar.zip` lớn, tải chậm hoặc render nặng. Do đó, output cho WebAR phải cân bằng giữa độ đúng hình học và độ nhẹ.

Trong ngữ cảnh này, "mesh tốt" không đồng nghĩa với "mesh nhiều chi tiết nhất". Mesh tốt là mesh có đủ chi tiết ở vùng ảnh hưởng đến occlusion hoặc navigation, nhưng không tạo quá nhiều tam giác ở vùng ít quan trọng. Với occlusion, nhiều chi tiết nhỏ có thể không cần nếu vật thể ảo chỉ cần bị che đúng ở mức hình dáng tổng quát. Với navigation, mesh cần sạch, liên thông và có mặt walkable rõ hơn là mô tả mọi chi tiết nhỏ của bề mặt.

WebAR cũng yêu cầu artifact có thể đóng gói và tái mở dễ dàng. Một pipeline nghiên cứu chỉ xuất mesh rồi là chưa đủ; runtime cần biết file nào là scene render, file nào là occlusion mesh, có navmesh hay không, metric và warning nằm ở đâu. Vì vậy, ngoài thuật toán tái tạo geometry, hệ thống còn cần hợp đồng output rõ ràng để viewer hoặc công cụ tích hợp đọc được.

2.5. Voxelization và mesh extraction

Một cách tự nhiên để chuyển dữ liệu 3DGS sang geometry proxy là voxelization. Voxelization rời rạc hóa không gian thành lưới ô 3D và đánh dấu ô nào có khả năng bị chiếm dụng. Với 3DGS, trạng thái occupied của voxel có thể được suy ra từ đóng góp opacity của các Gaussian lân cận. Khi occupancy grid đã được tạo, hệ thống có thể fill, carve hoặc trích xuất bề mặt.

Ưu điểm của voxelization là ổn định và dễ kiểm soát. Nó không đòi hỏi dữ liệu đầu vào phải có topology hoặc normal tốt. Nó cũng phù hợp với các thao tác hình học sau đó như fill vùng rỗng, carve theo capsule agent hoặc crop vùng occupied. Nhược điểm là phụ thuộc vào kích thước voxel. Voxel quá lớn làm mất chi tiết và tạo occlusion thô; voxel quá nhỏ làm tăng thời gian xử lý, bộ nhớ và số tam giác.

Sau voxelization, mesh extraction chuyển biên giữa vùng occupied và empty thành mesh. Marching Cubes là thuật toán kinh điển cho bài toán dựng bề mặt từ scalar field hoặc occupancy grid [3]. Thuật toán duyệt từng cell, xét trạng thái các đỉnh và dùng bảng tra để sinh tam giác xấp xỉ mặt iso-surface. Trong đề tài, hướng này phù hợp cho occlusion mesh vì nó biến dữ liệu rời rạc thành GLB có thể dùng trong runtime.

2.6. Reconstruction liên quan: SuGaR và Poisson

Ngoài voxelization, có các hướng reconstruction khác có thể chuyển dữ liệu hoặc điểm 3D thành mesh. SuGaR là phương pháp liên quan trực tiếp đến 3DGS, khai thác quan hệ giữa Gaussian và bề mặt để dựng mesh có độ trung thực cao hơn trong các cảnh phù hợp [2]. Poisson Surface

Reconstruction là hướng cổ điển hơn, dựng bề mặt từ point set có normal bằng cách giải bài toán trường vector/indicator function [4].

Trong báo cáo này, SuGaR và Poisson được xem là các phương pháp liên quan và là hướng adapter của hệ thống, không phải trọng tâm lý thuyết chính. Điểm quan trọng là chúng bổ sung cho voxel path. Voxel path có ưu điểm kiểm soát tốt, dễ tích hợp vào Rust core và phù hợp cho geometry proxy. Adapter SuGaR/Poisson mở ra khả năng dùng thuật toán ngoài khi cần chất lượng surface reconstruction khác, miễn là adapter trả về mesh theo hợp đồng mà pipeline có thể xử lý tiếp.

2.7. Navigation mesh và Recast

Navigation mesh là biểu diễn rút gọn của vùng có thể di chuyển. Thay vì tìm đường trên toàn bộ mesh chi tiết, runtime tìm đường trên tập polygon đã được lọc theo điều kiện walkable. Recast Navigation là một bộ công cụ phổ biến cho bài toán này [5]; repository dùng rerecast, wrapper Rust của Recast [6]. Các tham số như agentHeight, agentRadius, maxSlopeDegrees và walkableClimb quyết định vùng nào được coi là đi được.

Trong AR, navmesh có vai trò khác occlusion mesh nhưng quan trọng tương đương nếu có agent hoặc logic di chuyển. Occlusion mesh trả lời câu hỏi vật thể ảo có bị cảnh thật che không. Navigation mesh trả lời câu hỏi tác tử ảo có thể đi ở đâu. Hai loại mesh có thể cùng xuất phát từ collision mesh, nhưng tiêu chí đánh giá khác nhau. Một collision mesh có nhiều chi tiết nhỏ có thể gây nhiễu cho navmesh; ngược lại một navmesh sạch có thể bỏ qua nhiều chi tiết không cần thiết cho occlusion.

2.8. Nền tảng WebAR và desktop

Ứng dụng desktop dùng Tauri làm shell, React/Vite cho frontend và PlayCanvas cho preview/render phía web [7, 9]. Tauri phù hợp vì cho phép giao diện web gọi command Rust cục bộ mà không cần server riêng. PlayCanvas phù hợp cho preview và WebAR viewer vì là engine WebGL/WebGPU-oriented, dễ đóng gói vào bundle HTML. <model-viewer> là tham chiếu quan trọng trong hệ sinh thái web cho cách hiển thị asset 3D/AR theo hướng đơn giản hóa tích hợp [10].

Với xử lý nặng, Rust core dựa vào CPU và GPU path. GPU compute dùng wgpu, một abstraction portable theo hướng WebGPU [8]. Cách tổ chức này phản ánh yêu cầu của đề tài: giao diện cần đủ thuận tiện để thao tác dữ liệu, nhưng xử lý chính phải nằm ở core có thể kiểm thử, benchmark và tái sử dụng.

3DGS data	render	Tối ưu cho hiển thị, lưu Gaussian, opacity và màu.
Geometry proxy		Tối ưu cho AR interaction, gồm occlusion mesh và navigation mesh.
WebAR bundle		Tối ưu cho phân phối, cần nhẹ, tự chứa và có manifest.

Hình 2: Ba lớp dữ liệu cần phân biệt khi đưa 3DGS vào AR.

3. Phân tích yêu cầu

3.1. Từ khoảng trống 3DGS đến yêu cầu hệ thống

Yêu cầu của đề tài không nên được hiểu như danh sách chức năng rời rạc. Chúng xuất phát từ một khoảng trống cụ thể: 3DGS có thể render cảnh thật rất thuyết phục, nhưng chưa cung cấp dữ liệu hình học mà hệ AR cần để tương tác với cảnh. Vì vậy, yêu cầu hệ thống phải đi từ bản chất của bài toán AR: cần nhận biết vật cản, cần vùng đi lại, cần tọa độ và tỷ lệ hợp lý, cần output đủ nhẹ để chạy trên WebAR, và cần pipeline đủ tái lập để kiểm thử.

Nếu chỉ dừng ở việc đọc file 3DGS và xuất một mesh, công cụ sẽ chưa giải quyết trọn vẹn bài toán. Mesh có thể sai tỷ lệ, quá nặng, không dùng được cho occlusion, không sinh được navmesh, hoặc không đóng gói được thành artifact mà viewer WebAR đọc được. Do đó, yêu cầu của đề tài được phân tích theo các nhóm năng lực thay vì chỉ theo nút bấm hoặc module.

3.2. Nhóm yêu cầu theo luận điểm nghiên cứu

Bảng 1: Các nhóm yêu cầu suy ra từ bài toán 3DGS phục vụ AR.

Nhóm yêu cầu	Vấn đề cần giải quyết	Hàm ý đối với hệ thống
Geometry proxy	3DGS thiếu surface/collision rõ ràng dù render tốt.	Phải sinh geometry proxy từ .ply/.splat, không chỉ giữ dữ liệu render.
Occlusion	AR cần vật thể thật che đúng vật thể ảo.	Phải xuất occlusion mesh ở định dạng phổ biến như GLB.
Navigation	Agent hoặc logic di chuyển cần vùng walkable.	Phải có khả năng bake navmesh với tham số agent và cảnh báo khi rỗng.
Scale/coordinate	Scene tái tạo có thể sai scale, sai up axis hoặc lệch transform.	Phải có recipe căn chỉnh scale/up axis và bake transform vào dữ liệu.
WebAR bundle	Runtime web cần artifact nhẹ, dễ mở và tự chứa.	Phải đóng gói scene.sog, mesh, viewer và metadata thành bundle.
Performance	Cảnh 3DGS có thể có rất nhiều splat, mesh quá nặng sẽ khó dùng.	Phải có GPU path, metric thời gian, triangle count và tỷ lệ dung lượng.
Reproducibility	Kết quả cần kiểm tra lại được, không phụ thuộc thao tác thủ công khó truy vết.	Phải lưu config, recipe, manifest, warning và metric.
Extensibility	Không một thuật toán reconstruction phù hợp mọi cảnh.	Phải hỗ trợ voxel path và adapter path cho SuGaR/Poisson.

Cách nhóm yêu cầu này phản ánh đúng bản chất nghiên cứu của đề tài. Geometry aware-

ness là yêu cầu gốc. Occlusion readiness và navigation readiness là hai yêu cầu AR trực tiếp. WebAR deliverability và lightweight output là yêu cầu triển khai thực tế. Reproducibility bảo đảm báo cáo và demo có thể được kiểm chứng thay vì chỉ mô tả bằng quan sát.

3.3. Geometry awareness

Yêu cầu đầu tiên là hệ thống phải chuyển dữ liệu 3DGS thành dữ liệu hình học. Dữ liệu hình học ở đây không nhất thiết là reconstruction surface hoàn hảo cho mục đích trình diễn, mà là geometry proxy phục vụ AR. Điều này làm thay đổi tiêu chí đánh giá: mesh cần đúng vị trí, đúng tỷ lệ, đủ kín hoặc đủ ổn định cho occlusion/collision, và không quá nặng. Một mesh có visual detail cao nhưng rỗng, lệch scale hoặc quá nhiều tam giác vẫn không đạt yêu cầu AR.

Từ yêu cầu này, hệ thống cần các bước đọc nguồn, chuẩn hóa dữ liệu splat, lọc dữ liệu lỗi, voxel hóa hoặc dùng adapter reconstruction, rồi xuất mesh. Các bước này phải làm việc với hai định dạng đầu vào chính là .ply và .splat. Đầu vào không được giả định là sạch tuyệt đối, vì dữ liệu 3DGS thực tế có thể có splat mờ, floater hoặc cụm nhiễu.

3.4. Occlusion readiness

Occlusion là một trong những yêu cầu cốt lõi của AR. Khi vật thể ảo được đặt vào cảnh thật, người dùng kỳ vọng vật thể đó bị môi trường thật che khuất đúng. Nếu không có occlusion mesh, vật thể ảo dễ nổi lên trên ảnh camera hoặc xuyên qua vật thật, làm mất cảm giác hiện diện. Vì vậy, hệ thống phải sinh được occlusion mesh ở định dạng runtime có thể đọc, cụ thể là GLB.

Occlusion mesh không cần chứa texture hay vật liệu phức tạp. Nó chủ yếu cần bề mặt hình học. Tuy nhiên, nó phải đủ nhẹ để WebAR tải được và đủ ổn định để không gây artifact che khuất quá nhiều hoặc quá ít. Điều này dẫn đến yêu cầu có metric về số tam giác, kích thước file, geometric error và warning khi mesh rỗng. Các metric này giúp người vận hành đánh giá output thay vì chỉ thấy pipeline báo "done".

3.5. Navigation readiness

Navigation mesh là yêu cầu riêng so với occlusion. Nó không chỉ cần mesh tồn tại, mà cần vùng walkable hợp lý theo tham số agent. Một cảnh có thể có occlusion mesh tốt nhưng navmesh không dùng được nếu mặt đi lại bị chia cắt, bị chặn bởi voxel fill/carve sai hoặc up axis sai. Vì vậy, hệ thống cần cấu hình navmesh rõ ràng và cần cảnh báo khi navmesh sinh 0 tam giác.

Yêu cầu navigation cũng cho thấy giới hạn của cách tiếp cận thuần hình học. Hệ thống có thể suy luận vùng walkable theo độ dốc, chiều cao và bán kính agent, nhưng chưa tự hiểu ngữ nghĩa vật thể. Do đó, trong phạm vi hiện tại, đề tài tập trung vào geometry processing; semantic understanding/segmentation chỉ nên được xem là hướng phát triển sau.

3.6. Scale và coordinate consistency

Một artifact AR chỉ có giá trị khi nó khớp hệ tọa độ và tỷ lệ với thế giới thật. Nếu scale sai, vật thể ảo có thể quá lớn hoặc quá nhỏ. Nếu up axis sai, navmesh có thể nằm trên tường thay vì sàn.

Nếu transform chỉ áp dụng trong preview nhưng không bake vào nguồn, scene render và mesh export sẽ lệch nhau. Vì vậy, hệ thống cần cơ chế căn chỉnh và bake transform nhất quán.

Trong codebase hiện tại, GUI dùng hai scale endpoints và khoảng cách thật để suy ra scale. Up axis được chọn từ danh sách trục. Scene transform gồm rotation XYZ và position XYZ được export vào nguồn tạm trước khi bake. Các quyết định này đáp ứng yêu cầu consistency ở mức thực dụng: người dùng có thể hiệu chỉnh cảnh mà không cần sửa file nguồn hoặc viết JSON thủ công.

3.7. WebAR deliverability và hiệu năng

Đầu ra của đề tài hướng tới WebAR nên yêu cầu hiệu năng và độ nhẹ là trọng tâm. Một pipeline offline có thể chấp nhận mesh rất nặng, nhưng WebAR cần bundle tải được, mở được và chạy được trên thiết bị có tài nguyên hạn chế. Vì vậy, hệ thống cần theo dõi kích thước `scene.sog`, `occlusion.glb`, `webar.zip`, số tam giác và thời gian xử lý.

Hiệu năng không chỉ là tốc độ bake. Nó còn liên quan đến kích thước artifact và độ phức tạp runtime. GPU voxelization là đường chính để xử lý cảnh lớn nhanh hơn, nhưng CPU path vẫn cần tồn tại cho kiểm thử, fallback và so sánh parity. Cấu trúc metric trong manifest giúp đánh giá trade-off này: voxel size nhỏ hơn có thể tăng độ chi tiết nhưng làm mesh nặng hơn; backend GPU có thể nhanh hơn nhưng cần kiểm tra sai khác với CPU khi bật compare.

3.8. Tính tái lập và khả năng kiểm chứng

Một yêu cầu quan trọng của báo cáo nghiên cứu là kết quả phải có thể tái lập. Với pipeline xử lý 3D, nếu chỉ ghi rằng "kết quả tốt" mà không lưu config, recipe và metric thì rất khó kiểm chứng. Vì vậy, mỗi lượt bake cần để lại dấu vết: input path, source stats, alignment, process config, artifact list, timing, triangle count, size ratio và warning.

Manifest đóng vai trò trung tâm cho yêu cầu này. Nó không chỉ phục vụ viewer mà còn phục vụ đánh giá. Khi có benchmark chính thức, bảng kết quả có thể được trích từ manifest hoặc summary CSV thay vì nhập tay. Điều này đặc biệt quan trọng vì phần thực nghiệm định lượng của đề tài chưa chốt; báo cáo cần giữ ranh giới rõ giữa cơ chế đo đã có và số liệu chưa được xác minh.

3.9. Phạm vi yêu cầu

Trong phạm vi hiện tại, đề tài không đặt mục tiêu xây dựng một hệ AR production đầy đủ. Hệ thống tập trung vào công cụ xử lý dữ liệu 3DGS thành artifact hình học phục vụ AR. Các chức năng như semantic segmentation, nhận diện vật thể, tối ưu runtime đa thiết bị hoặc authoring scene phức tạp nằm ngoài phạm vi chính. Việc giới hạn phạm vi này giúp đề tài tập trung vào khoảng trống quan trọng nhất: từ 3DGS render data sang geometry proxy và WebAR bundle có thể kiểm chứng.

4. Thiết kế hệ thống

4.1. Quan điểm thiết kế theo dataflow

Thiết kế hệ thống được nhìn theo luồng dữ liệu logic thay vì theo từng màn hình giao diện. Một lượt xử lý bắt đầu từ dữ liệu 3DGS đầu vào, đi qua recipe căn chỉnh, process config, reconstruction stage và cuối cùng là export bundle. Cách nhìn này phù hợp với mục tiêu nghiên cứu vì mỗi boundary trong pipeline đều trả lời một câu hỏi riêng:

- Input chứa dữ liệu gì và đã được chuẩn hóa chưa?
- Calibration recipe mô tả cách đưa scene về hệ tọa độ sử dụng được trong AR như thế nào?
- Processing config quyết định thuật toán và tham số xử lý ra sao?
- Reconstruction stage tạo geometry proxy bằng đường nào?
- Export stage đóng gói kết quả để runtime và đánh giá dùng được như thế nào?

Input	.ply/.splat được đọc và chuẩn hóa thành bảng splat nội bộ.
Calibration Recipe	Scale endpoints, scale distance, up axis, origin và edit recipe.
Process Config	Reconstruction method, voxel backend, fill, carve, mesh, navmesh, export.
Reconstruction	Voxel path trong Rust core hoặc adapter path cho SuGaR/Poisson.
Export	Scene render, geometry proxy, manifest và WebAR bundle.

Hình 3: Dataflow logic của một lượt xử lý 3DGS-to-AR.

Luồng này cũng giúp tách trách nhiệm. Calibration không quyết định thuật toán reconstruction. Reconstruction không quyết định người dùng đã chọn scale như thế nào. Export không cần biết chi tiết GUI. Nhờ vậy, cùng một core có thể phục vụ GUI, CLI, test và benchmark.

4.2. Input boundary

Input boundary chịu trách nhiệm đưa dữ liệu nguồn về một biểu diễn thống nhất. Người dùng có thể cung cấp .ply hoặc .splat; mỗi định dạng có cách lưu thuộc tính khác nhau. Nếu các stage sau phải xử lý trực tiếp từng định dạng, pipeline sẽ khó bảo trì và dễ phát sinh lỗi không đồng nhất. Vì vậy, core đọc nguồn và chuẩn hóa thành bảng splat nội bộ trước khi alignment, filter hoặc reconstruction chạy.

Boundary này có hai yêu cầu quan trọng. Thứ nhất, nó phải giữ đủ thông tin cho cả render và geometry: vị trí, scale, rotation, opacity và màu. Thứ hai, nó phải ghi nhận source stats như format, số splat và kích thước nguồn để manifest có thể phục vụ đánh giá. Input boundary vì vậy không chỉ là file parser, mà là điểm bắt đầu của tính tái lập.

4.3. Calibration Recipe boundary

Calibration Recipe mô tả các quyết định gắn với nguồn dữ liệu và thao tác căn chỉnh. Nó gồm up axis, floor normal, scale points, scale distance meters, origin và edit operations. Đây là boundary giữa dữ liệu thô và dữ liệu có ý nghĩa trong hệ tọa độ AR. Nếu boundary này không rõ, các stage sau sẽ khó biết dữ liệu đã được scale chưa, đã xoay chưa hoặc origin có được đặt lại không.

```
{
  "alignmentRecipe": {
    "upAxis": "y",
    "floorNormal": [0, 1, 0],
    "scalePoints": [[0, 0, 0], [2, 0, 0]],
    "scaleDistanceMeters": 2.0,
    "origin": [0, 0, 0]
  },
  "editRecipe": { "operations": [] }
}
```

Trong GUI hiện tại, floorNormal được suy ra từ up axis. Điều này cần được hiểu đúng: hệ thống đang cung cấp cơ chế chọn hướng trục lên và bake alignment, chưa phải thuật toán tự động fit sàn từ point cloud. Boundary recipe vẫn hữu ích vì nó làm quyết định căn chỉnh trở nên rõ ràng, có thể serialize, có thể kiểm thử và có thể tái chạy.

4.4. Processing Config boundary

Processing Config mô tả cách xử lý dữ liệu sau khi đã có recipe. Nó gồm reconstruction method, voxel config, fill mode, carve config, mesh mode, navmesh config và export config. Tách config khỏi recipe là quyết định quan trọng. Recipe trả lời câu hỏi "scene này được căn chỉnh như thế nào"; config trả lời câu hỏi "pipeline này xử lý scene bằng thuật toán và tham số nào".

Sự tách biệt này cho phép cùng một scene và cùng một recipe được chạy với nhiều cấu hình khác nhau. Ví dụ, có thể giữ scale/up axis không đổi nhưng so sánh backend CPU/GPU, so sánh voxel size, bật/tắt navmesh hoặc chuyển từ voxel sang adapter SuGaR/Poisson. Đây là cơ sở để benchmark và debug có ý nghĩa.

Bảng 2: Ranh giới giữa recipe và process config.

Boundary	Nội dung chính	Lý do tách riêng
Calibration Recipe	Scale points, distance, up axis, origin, edit operations	Gắn với scene và thao tác căn chỉnh; cần tái lập theo dữ liệu đầu vào.
Processing Config	Voxel backend, reconstruction method, fill/carve, mesh, navmesh, export	Gắn với thuật toán xử lý; cần thay đổi để thử nghiệm và tối ưu.

4.5. Reconstruction boundary

Reconstruction boundary là nơi pipeline chuyển từ splat table sang geometry proxy. Hệ thống có hai nhánh chính. Nhánh thứ nhất là voxel path trong Rust core: dữ liệu được voxel hóa,

fill/carve, trích xuất mesh và bake navmesh nếu bật. Nhánh thứ hai là adapter path: core ghi PLY đã lọc, gọi external adapter cho SuGaR hoặc Poisson, nhận mesh JSON và validate trước khi export.

Voxel path	Core tự xử lý: voxelization, fill, carve, mesh extraction, navmesh.
Adapter path	Core chuẩn bị input và gọi adapter SuGaR/Poisson qua JSON contract.
Common output	Mesh hợp lệ được đưa vào cùng export path và cùng manifest schema.

Hình 4: Hai nhánh reconstruction cùng hội tụ về geometry output.

Thiết kế này tránh khóa hệ thống vào một thuật toán duy nhất. Voxel path ổn định, dễ kiểm thử và phù hợp với geometry proxy. Adapter path cho phép thử các phương pháp reconstruction ngoài mà không làm core phụ thuộc vào môi trường Python/CUDA hoặc tool chuyên dụng. Điểm hội tụ là mesh hợp lệ: sau reconstruction, các bước export và đánh giá không cần biết mesh đến từ voxel hay adapter.

4.6. Export boundary

Export boundary chuyển kết quả xử lý thành artifact dùng được. Đây là điểm pipeline rời khỏi không gian thuật toán và bước vào không gian tích hợp. Output tối thiểu gồm dữ liệu render, geometry proxy, metadata và bundle. Tuy nhiên, phần này không chỉ là ghi file. Export phải bảo đảm manifest mô tả đúng artifact hiện có, xóa artifact stale khi navmesh không được sinh, ghi metric kích thước và warning để người dùng biết chất lượng output.

Trong dataflow tổng thể, manifest là sợi dây nối giữa xử lý, runtime và đánh giá. Viewer cần manifest để biết file nào cần load. Báo cáo cần manifest để trích số liệu. Người phát triển cần manifest để debug warning. Vì vậy, export boundary phải được xem là một phần của thiết kế hệ thống, không phải bước phụ sau thuật toán.

4.7. Các bất biến của pipeline

Để pipeline đáng tin cậy, mỗi boundary cần giữ một số bất biến. Input phải được parse thành bảng splat hợp lệ. Recipe phải chỉ chứa giá trị hữu hạn và scale points hợp lệ. Config phải chọn method và tham số có thể deserialize ở Rust core. Reconstruction phải trả mesh hợp lệ hoặc warning/lỗi rõ ràng. Export phải tạo manifest nhất quán với file thật trong output directory.

Bảng 3: Bất biến chính trong dataflow.

Stage	Bất biến cần giữ	Rủi ro nếu vi phạm
Input	Splat table có số cột khớp, giá trị cơ bản đọc được.	Pipeline lỗi muộn hoặc mesh rỗng khó debug.
Recipe	Scale distance dương, scale points hữu hạn, up axis hợp lệ.	Artifact sai tỷ lệ, sai hướng hoặc không bake được.

Stage	Bất biến cần giữ	Rủi ro nếu vi phạm
Config	Method, backend, fill/carve/-navmesh deserialize đúng.	GUI và core hiểu khác nhau về cùng một lượt bake.
Reconstruction	Mesh output có vertex/index hữu hạn và triangle count hợp lý.	GLB lỗi, occlusion sai hoặc navmesh không sinh được.
Export	Manifest khớp file thật, không giữ stale artifact.	Viewer đọc nhầm file hoặc báo cáo dùng metric sai.

Các bất biến này giải thích vì sao hệ thống cần test ở nhiều lớp. Unit test kiểm tra từng stage. E2E test kiểm tra GUI serialize recipe/config đúng. Benchmark kiểm tra metric và hiệu năng. Nếu chỉ test một lớp, nhiều lỗi dataflow sẽ không lộ ra.

4.8. Lý do thiết kế phù hợp với đề tài

Thiết kế theo dataflow phù hợp với mục tiêu nghiên cứu vì nó làm rõ chuyển đổi từ dữ liệu render sang dữ liệu AR. Mỗi boundary tương ứng với một quyết định có thể giải thích trong báo cáo: chuẩn hóa đầu vào, căn chỉnh thế giới thật, chọn thuật toán, dựng geometry, đóng gói artifact. Điều này cũng giúp tránh mô tả hệ thống như một tập module rời rạc. Thay vào đó, báo cáo có thể chứng minh rằng các thành phần đều phục vụ cùng một luồng: biến 3DGS thành dữ liệu WebAR có thể dùng và có thể kiểm chứng.

5. Phương pháp xử lý dữ liệu

5.1. Chuẩn hóa bảng dữ liệu Splat

Sau khi đọc file nguồn, hệ thống đưa dữ liệu về bảng splat nội bộ. Bảng này lưu các thuộc tính cần thiết cho các bước sau: vị trí, scale, rotation, opacity và màu. Việc chuẩn hóa giúp filter, alignment, voxelization và export không cần biết dữ liệu đến từ .ply hay .splat. Đây là lựa chọn quan trọng vì mỗi định dạng có cách lưu property khác nhau, trong khi pipeline downstream cần một API thống nhất.

Ở mức xử lý hình học, thuộc tính quan trọng nhất là vị trí và scale/rotation của Gaussian. Vị trí quyết định bounding box, scale calibration, voxel coverage và mesh extraction. Rotation và scale của Gaussian ảnh hưởng cách ước lượng đóng góp opacity lên voxel. Opacity giúp loại bỏ splat quá mờ và quyết định voxel nào được coi là solid. Màu chủ yếu phục vụ render/export scene.sog, nhưng không phải yếu tố chính khi tạo collision mesh.

5.2. Alignment bake

Alignment bake áp dụng scale, rotation và origin vào dữ liệu trước khi các bước filter và reconstruction chạy. Nếu recipe có hai scale points và khoảng cách vật lý, backend tính khoảng cách hiện tại trong scene, sau đó suy ra hệ số scale. Nếu có floor normal và up axis, backend tính rotation đưa normal về hướng up axis. Với GUI hiện tại, floor normal đã bằng vector của up

axis nên rotation alignment thường là identity; phần xoay thực tế do người dùng chỉnh ở scene transform trước đó.

Quy trình alignment có thể tóm tắt:

$$s = \frac{d_{real}}{\|p_1 - p_0\|}$$

Trong đó p_0 và p_1 là hai scale endpoints, d_{real} là khoảng cách thật do người dùng nhập. Sau khi tính transform, hệ thống áp dụng lên vị trí, rotation và scale của từng splat. Việc bake vào dữ liệu đảm bảo artifact render và geometry cùng nằm trong một hệ tọa độ.

5.3. Scene transform trong desktop app

Scene transform là lớp chỉnh sửa trước alignment. Người dùng thao tác position XYZ và rotation XYZ trong editor. Khi bấm Bake, frontend gọi `export_edited_source`; Tauri đọc nguồn, áp dụng transform lên vị trí và quaternion rotation của splat, rồi ghi PLY tạm. Pipeline xử lý file tạm này như nguồn đầu vào. Cách này có hai ưu điểm: preview và output khớp nhau, và core không cần phụ thuộc trạng thái UI.

Về mặt toán học, nếu R_e là rotation từ Euler XYZ và t_e là translation, mỗi điểm nguồn x được biến đổi thành:

$$x' = R_e x + t_e$$

Rotation của Gaussian cũng được compose với R_e để hướng ellipsoid không bị lệch so với vị trí mới. Đây là điểm quan trọng vì chỉ dịch/xoay tâm Gaussian mà không xoay covariance sẽ làm biểu diễn không nhất quán.

5.4. Lọc dữ liệu

Filtering loại bỏ dữ liệu lỗi hoặc không cần thiết trước khi reconstruction. Core hiện có các nhóm filter sau:

- `filter_nan`: loại splat có giá trị không hữu hạn.
- `filter_opacity_min`: loại splat có opacity dưới ngưỡng.
- `filter_box`: giữ hoặc loại theo bounding box.
- `filter_sphere`: thao tác theo vùng cầu.
- `filter_cluster`: tìm cụm liên thông quanh seed trong voxel thô.
- `filter_floaters_by_voxel_contribution`: loại splat có đóng góp voxel thấp.

Trong GUI profile mặc định, `editRecipe.operations` đang rỗng để tránh tự động cắt nhằm dữ liệu khi người dùng chưa cấu hình. Tuy nhiên, core vẫn giữ filter nâng cao để CLI hoặc workflow sau này sử dụng. Thiết kế này thận trọng: với dữ liệu 3DGS, một filter quá mạnh có thể loại mất bề mặt mỏng hoặc chi tiết quan trọng.

5.5. Voxelization CPU/GPU

Voxelization nhận bảng splat và sinh occupancy grid. Mỗi Gaussian đóng góp vào các voxel lân cận tùy theo vị trí, scale, rotation và opacity. Voxel được đánh dấu solid nếu giá trị tích

lũy vượt `opacityThreshold`. Cấu hình chính gồm `voxel.size`, `opacityThreshold`, `backend` và `compareCpuGpu`.

CPU backend là baseline để debug. GPU backend dùng `wgpu` để tăng throughput cho cảnh lớn. GUI hiện sinh backend: "gpu" làm mặc định. Trong core, nếu bật `compareCpuGpu`, hệ thống vẫn tính cả hai grid và ghi số mismatch vào manifest. Cách này cho phép kiểm chứng rằng GPU path không thay đổi ý nghĩa thuật toán so với CPU path.

Bảng 4: So sánh vai trò CPU và GPU path.

Path	Vai trò	Ghi chú
CPU	Baseline, kiểm thử, fallback cấu hình	Dễ debug, không phụ thuộc adapter GPU.
GPU	Đường chính cho GUI bake	Phù hợp cảnh nhiều splat; metric ghi <code>gpuVoxelMs</code> .
Compare	Kiểm chứng sai khác	Ghi <code>cpuGpuVoxelMismatches</code> khi bật.

5.6. Fill và carve

Occupancy grid sau voxelization có thể chứa lỗ, vùng hỏ hoặc vùng solid không phù hợp với navigation. Fill và carve bổ sung logic hình học theo profile. `floor-fill` ưu tiên lấp theo mặt nền. `exterior-fill` hỗ trợ cảnh trong phòng, nơi cần phân biệt trong/ngoài khối chiếm dụng. `none` giữ nguyên grid khi profile không cần suy diễn thêm.

Carve dùng capsule đại diện cho agent với `agentHeight` và `agentRadius`. Từ seed position, thuật toán xác định vùng agent có thể đi qua và loại các solid cell không phù hợp với không gian di chuyển. Nếu seed nằm sai hoặc bị chặn, manifest có thể ghi warning. Đây là lý do profile `interior-room` bật carve còn `object-prop` tắt: một vật thể nhỏ không cần vùng đi lại.

5.7. Mesh extraction

Sau fill/carve, hệ thống crop grid về vùng occupied để giảm kích thước xử lý rồi trích xuất mesh. Chế độ `faces` sinh mặt theo biên voxel, phù hợp collision proxy đơn giản. Chế độ `smooth` dùng Marching Cubes để tạo bề mặt mượt hơn. Sau khi sinh mesh, hệ thống validate, đếm số tam giác và ghi metric. Nếu mesh có 0 tam giác, manifest ghi warning để người dùng kiểm tra voxel size, opacity threshold, profile hoặc seed.

Output mesh được ghi ở hai dạng. `collision_mesh.json` giúp test và debug vì dễ đọc bằng code. `occlusion.glb` là artifact chuẩn để viewer hoặc engine khác sử dụng. Việc xuất GLB giúp giảm rào cản tích hợp với WebAR, game engine hoặc công cụ 3D phổ biến.

5.8. Adapter SuGaR/Poisson

Khi reconstruction method là `sugar` hoặc `poisson`, pipeline không chạy voxel mesh extraction làm kết quả chính. Thay vào đó, core ghi PLY đã lọc vào `adapter-work`, gửi request JSON cho adapter command và nhận mesh JSON trả về. Adapter có timeout để tránh treo bake vô hạn. Backend kiểm tra đường dẫn mesh nằm trong output directory sau khi canonicalize, nhằm giảm rủi ro adapter ghi file ngoài vùng dự kiến.

Thiết kế adapter đặc biệt hữu ích cho nghiên cứu vì SuGaR và Poisson có yêu cầu môi trường khác nhau. SuGaR thường đi cùng pipeline tối ưu hóa riêng, còn Poisson cần point/normal phù hợp. Nếu ép tất cả vào một binary desktop, quá trình build, đóng gói và debug sẽ nặng. Adapter contract cho phép tiến hóa từng thuật toán độc lập nhưng vẫn giữ chung artifact cuối.

5.9. Navigation mesh bằng rerecast

Navmesh được bake từ collision mesh sau reconstruction. Core gọi rerecast với tham số agent và walkability. Nếu navmesh disabled, pipeline không xuất `navmesh.glb` hoặc `navmesh.bin`. Nếu enabled nhưng không sinh được tam giác, manifest ghi warning. Điều này rất quan trọng trong demo AR: navmesh rỗng không nên bị hiểu nhầm là không có lỗi, vì nguyên nhân có thể là up axis sai, seed carve bị chặn hoặc mesh không có mặt walkable.

Navmesh không phải artifact bắt buộc cho mọi scene. Với object prop, navmesh thường không có ý nghĩa. Với interior room, navmesh giúp mô phỏng vùng đi lại hoặc đặt tác tử ảo. Với outdoor terrain, profile hiện tắt navmesh mặc định vì địa hình thực có thể cần chiến lược riêng để phân biệt bề mặt đi được, vật cản thấp và vùng không ổn định.

5.10. Export và đóng gói

Sau khi có scene data và mesh, pipeline export các file vào output directory. `scene.sog` lưu dữ liệu render đã alignment. `occlusion.glb` lưu mesh. `index.html` và PlayCanvas runtime được ghi để viewer có thể mở scene. `manifest.json` được ghi nhiều lần trong quá trình tạo `webar.zip` để đảm bảo size của zip trong manifest khớp với file thực tế.

Stage 1	Decode source và chuẩn hóa splat table.
Stage 2	Bake scene transform, scale, up axis và origin.
Stage 3	Filter NaN/opacity/region/cluster/floater nếu recipe yêu cầu.
Stage 4	Voxel CPU/GPU hoặc external reconstruction adapter.
Stage 5	Fill, carve, mesh extraction và navmesh.
Stage 6	Export GLB, SOG, manifest, viewer và WebAR ZIP.

Hình 5: Các stage xử lý chính trong pipeline.

6. Triển khai ứng dụng desktop

6.1. Vai trò của desktop app

Ứng dụng desktop trong `apps/desktop` là giao diện cuối cùng của đề tài. CLI vẫn quan trọng cho proof of concept, benchmark và automation, nhưng không phải trải nghiệm chính cho người dùng. Desktop app giải quyết vấn đề mà CLI khó làm tốt: preview trực quan, chọn điểm scale trên scene, chỉnh rotation/position, theo dõi progress, xem manifest và lưu bundle. Với dữ liệu 3DGS, việc nhìn trực tiếp scene trước khi bake là cần thiết vì sai up axis hoặc sai scale rất khó phát hiện chỉ qua JSON.

Frontend được viết bằng React, build bằng Vite. Preview dùng PlayCanvas để hiển thị source. Backend desktop dùng Tauri command để gọi Rust core và thao tác file cục bộ. Kiến trúc này cho phép ứng dụng có UI web hiện đại nhưng vẫn truy cập filesystem và xử lý nặng bằng Rust.

6.2. Workflow giao diện

Workflow chính của GUI gồm các trạng thái: chọn nguồn, load, preview/edit, calibration, bake, export. Người dùng nhập Source PLY/SPLAT Path và Output Directory, bấm Load, sau đó canvas hiển thị preview. Khi source đã load, người dùng có thể double-click hoặc dùng picker để đặt `scale0` và `scale1`. Input Scale Calibration Distance mô tả khoảng cách thật giữa hai điểm. Up Axis được chọn từ danh sách trục. Nếu chọn reconstruction method là SuGaR hoặc Poisson, Adapter Command trở thành điều kiện bắt buộc.

Nút Bake Geometry chỉ enabled khi đủ điều kiện. Các điều kiện gồm source sẵn sàng, scale endpoints do người dùng chọn, distance hợp lệ, adapter command hợp lệ nếu cần, và scene không bị ẩn/xóa. Khi bake chạy, progress stage được cập nhật theo event từ Tauri, ví dụ decode, alignment, filters, voxelize, fill, carve, mesh, navmesh, export và done. Nút Cancel vẫn còn để người dùng dừng job dài.

Bảng 5: Các nhóm điều khiển chính trong GUI.

Nhóm	Điều khiển	Mục đích
Nguồn dữ liệu	Source path, Output directory, Load	Nạp <code>.ply/.splat</code> và chuẩn bị thư mục bake.
Editor	Position XYZ, Rotation XYZ	Chỉnh tư thế scene trước khi export nguồn tạm.
Calibration	Scale distance, pick target, up axis	Tạo recipe scale và hướng trục.
Processing	Reconstruction method, adapter command, bake profile	Chọn chiến lược tái tạo hình học.
Job controls	Bake, Cancel, Save ZIP	Chạy, hủy và lưu bundle.
Kết quả	Manifest, artifacts, warnings	Kiểm tra đầu ra và cảnh báo.

6.3. Quản lý calibration state

Hook `useCalibration` giữ các state quan trọng: distance, scale points, up axis, geometry profile, reconstruction method, adapter command và pick mode. Một số state được lưu vào `localStorage` để giữ lại giữa các phiên. `makeAlignmentRecipe` sinh recipe từ distance, scale points và up axis. `makeProcessConfig` sinh cấu hình bake từ profile, reconstruction method và adapter command.

Sau chỉnh sửa trong phiên làm báo cáo, `makeProcessConfig` mặc định dùng `voxel.backend = "gpu"` cho các profile GUI. Điều này khớp với định hướng GPU là đường chính. Rust core vẫn có enum `VoxelBackend::Cpu` và `VoxelBackend::Gpu`; config cũ hoặc CLI vẫn có thể chọn CPU khi cần. Unit test calibration và e2e test đã được cập nhật kỳ vọng backend GPU.

Một chi tiết cần nói rõ là `geometryProfile` không làm thay đổi `recipe alignment` trong code hiện tại. Tham số này được truyền vào `makeAlignmentRecipe` nhưng `recipe` vẫn chỉ gồm `up axis`, `floor normal`, `scale points`, `distance` và `origin`. Profile chủ yếu ảnh hưởng `process config`: `fill`, `carve` và `navmesh`.

6.4. Editor transform

Editor transform nằm trong domain editor, gồm `SceneTransformControls`, `sceneTransform` và hook quản lý scene. Người dùng nhập position XYZ và rotation XYZ. Khi `bake`, `prepareBakeSource` gọi `command export` để tạo nguồn đã chỉnh. Tauri `command validate transform` hữu hạn, đọc source, apply transform và ghi file PLY tạm. Test Rust kiểm tra transform cập nhật vị trí và rotation của splat.

Thiết kế này có ý nghĩa thực dụng. Thay vì yêu cầu người dùng xác định mặt sàn bằng nhiều điểm hoặc chỉnh trực tiếp file nguồn, GUI cung cấp transform toàn cục. Với các scene quét có orientation chưa đúng, rotation XYZ là cách nhanh để đưa scene về tư thế hợp lý. Với scene cần đặt vào hệ tọa độ AR, position XYZ giúp dịch về vị trí mong muốn trước khi pipeline xuất artifact.

6.5. Tauri command bridge

Tauri command bridge gồm các command chính:

- `load_source`: đọc metadata nguồn, trả path, bytes, format và số splat.
- `export_edited_source`: apply scene transform và ghi nguồn tạm.
- `process_job`: gọi Rust core pipeline với input path, config và recipe.
- `cancel_job`: yêu cầu dừng job đang chạy.
- `save_bundle`: copy `webar.zip` sang vị trí người dùng chọn.
- `open_webar_viewer`: mở viewer cho output khi cần.

Command `process_job` trả manifest về GUI. Nếu source đã qua editor export, source context được ghi vào manifest để truy vết file gốc, file edited, số splat và bytes của nguồn đã chỉnh. Điều này giúp quá trình debug minh bạch hơn: khi artifact bị lệch, người phát triển biết pipeline đã xử lý file nào.

6.6. Preview và tương tác 3D

`PlayCanvas` preview giúp người dùng thấy dữ liệu trước khi `bake`. Với raw splat lớn, app có cơ chế dùng preview `downsample` để giữ UI responsive. E2E test kiểm tra rằng nguồn lớn có thể load metadata và preview path mà vẫn bảo toàn số lượng splat gốc. Canvas cũng hỗ trợ thao tác đặt marker cho scale endpoints. Overlay calibration hiển thị line scale, grid và hướng up axis để người dùng hiểu recipe hiện tại.

Preview không thay thế kết quả `bake`. Nó là công cụ hỗ trợ tương tác. Artifact cuối vẫn do Rust core tạo. Tách preview khỏi core pipeline giúp giao diện mượt hơn và giảm nguy cơ logic render ảnh hưởng logic geometry.

6.7. Hiển thị kết quả

Sau khi bake xong, GUI hiển thị thông tin manifest. Người dùng có thể thấy số tam giác collision, đường dẫn artifact, warning và trạng thái save bundle. Các warning quan trọng gồm collision mesh 0 triangles, navmesh 0 triangles, seed carve bị resolve hoặc adapter output vượt target triangle. Hiển thị warning trực tiếp trong GUI giúp người vận hành không phải mở manifest.json thủ công để phát hiện lỗi.



Hình 6: Quan hệ giữa UI, Tauri command và Rust core trong desktop app.

6.8. Ý nghĩa triển khai

Triển khai desktop chứng minh pipeline không chỉ là thuật toán backend. Để dùng được trong AR, công cụ cần cho người vận hành nhìn scene, chỉnh tư thế, nhập scale và nhận bundle. Đây là phần làm cho đề tài gần với ứng dụng thực tế hơn: các thao tác khó chuẩn hóa hoàn toàn bằng script được đưa vào GUI, còn các bước nặng và cần kiểm thử vẫn nằm trong core.

7. Đánh giá và kiểm thử

7.1. Nguyên tắc đánh giá

Đánh giá công cụ 3DGS-to-AR cần phân biệt ba loại kết quả. Loại thứ nhất là kết quả chức năng: pipeline có đọc được nguồn, sinh artifact và manifest đúng hợp đồng hay không. Loại thứ hai là kết quả hình học: mesh có đủ ổn định, đúng tỷ lệ, không rỗng và phù hợp với mục đích occlusion/navmesh hay không. Loại thứ ba là kết quả hiệu năng: thời gian xử lý, dung lượng file và lợi ích GPU so với CPU. Báo cáo hiện tại chỉ trình bày phương pháp, chỉ số và kiểm thử có trong codebase; số liệu benchmark lớn chưa được chốt nên không đưa bảng tốc độ hoặc tỷ lệ tối ưu hóa như kết quả cuối.

Nguyên tắc này quan trọng vì report cũ đã có xu hướng nêu kết quả quá mạnh trong khi dữ liệu chưa đủ. Với đề tài nghiên cứu nghiêm túc, phần đánh giá phải nói rõ cái gì đã được kiểm thử, cái gì là cơ chế đo, và cái gì đang chờ chạy benchmark chính thức.

7.2. Kiểm thử Rust core

Rust core có các test cho từng module xử lý. Nhóm alignment kiểm tra scale calibration, floor alignment và origin. Nhóm filter kiểm tra loại dữ liệu NaN, opacity, vùng box/sphere, cluster và floater contribution. Nhóm voxel kiểm tra voxelization, fill và carve. Nhóm mesh kiểm tra extraction và merge. Nhóm navmesh kiểm tra trường hợp sàn phẳng và tường đứng. Nhóm pipeline kiểm tra export artifact tùy chọn, seed sau alignment và profile object giữ splat.

Các test này bảo vệ logic nền tảng. Nếu alignment sai, mọi artifact phía sau sẽ lệch. Nếu filter sai, dữ liệu có thể bị mất hoặc giữ nhiều quá nhiều. Nếu mesh/navmesh sai, GUI vẫn có thể báo bake done nhưng artifact không dùng được. Vì vậy kiểm thử core là lớp bảo vệ đầu tiên.

Bảng 6: Các nhóm kiểm thử core cần duy trì.

Nhóm	Mục tiêu	Rủi ro nếu thiếu
Alignment	Kiểm tra scale, up axis, origin và transform downstream	Artifact sai tỷ lệ hoặc sai hướng.
Filters	Kiểm tra loại dữ liệu lỗi/nhiều	Mesh rỗng hoặc còn floater nhiều.
Voxel	Kiểm tra occupancy CPU/GPU, fill, carve	Collision mesh sai hoặc quá nặng.
Mesh	Kiểm tra faces/smooth extrac-tion	GLB lỗi, số tam giác bất thường.
NavMesh	Kiểm tra vùng walkable	Agent ảo không tìm đường được.
Pipeline	Kiểm tra export và manifest	Bundle thiếu file hoặc metadata sai.

7.3. Kiểm thử GUI

Frontend có unit test cho calibration profile. Test kiểm tra profile mặc định là object-prop, process config có voxel backend GPU, fill/carve/navmesh đúng theo profile, reconstruction method được normalize, và adapter command chỉ được yêu cầu với SuGaR/Poisson. Playwright e2e kiểm tra GUI serialize recipe và config đúng khi người dùng load source, đặt scale endpoints, chọn up axis, chọn reconstruction method, bake và save bundle.

E2E test cũng kiểm tra progress event và cancel state. Khi job đang chạy, Load và Bake bị khóa, Cancel còn enabled. Với source loading, test kiểm tra có thể hủy khi fetch hoặc decode đang pending. Với file lớn, test kiểm tra app có thể dùng preview path và không bật dialog xác nhận thừa. Những test này không đo chất lượng mesh, nhưng bảo vệ workflow người dùng và contract giữa GUI với Tauri.

7.4. Benchmark và metric

Thư mục docs/evaluation mô tả lệnh generate benchmark scene và chạy benchmark. CLI có thể tạo scene synthetic rồi chạy augmented-gaussian-cli benchmark. Config benchmark nằm ở docs/evaluation/benchmark-config.json. Khi chạy, output dự kiến gồm summary.json và summary.csv. Các metric được track gồm thời gian từng stage, backend voxel, CPU/GPU mismatch, kích thước source/SOG/GLB/ZIP, tỷ lệ dung lượng, số tam giác và geometric error.

```
cargo run -p augmented-gaussian-cli -- generate-bench-scenes --out target/bench-scenes
cargo run -p augmented-gaussian-cli -- benchmark \
  --input target/bench-scenes/bench-500000.splat \
  --out docs/evaluation/results/bench-500k \
  --config docs/evaluation/benchmark-config.json \
  --compare-cpu-gpu
```


Trong lần hoàn thiện báo cáo này, chưa chạy và chốt bộ số benchmark chính thức. Vì vậy các bảng kết quả định lượng để trống theo nghĩa học thuật: hệ thống có cơ chế đo, nhưng báo cáo không khẳng định tốc độ hoặc compression ratio khi chưa có dữ liệu được xác minh.

7.5. Chỉ số hình học

Manifest có nhóm geometric error gồm sample count, mean, RMS và P95. Ý nghĩa của nhóm này là so sánh mesh với splat centers ở mức xấp xỉ. Đây không phải metric hoàn hảo cho chất lượng occlusion, vì splat center không luôn nằm đúng trên bề mặt; tuy nhiên nó cung cấp tín hiệu định lượng ban đầu. Với benchmark chính thức, nên báo cáo geometric error cùng voxel size và triangle count để người đọc hiểu trade-off giữa độ chi tiết và kích thước mesh.

Một chỉ số quan trọng khác là số tam giác trước và sau merge. Nếu mesh có quá nhiều tam giác, WebAR có thể tải chậm hoặc render tốn tài nguyên. Nếu quá ít tam giác, occlusion có thể thô. Vì vậy không nên chỉ tối ưu một chiều theo kích thước file; phải nhìn đồng thời collision triangles, geometric error, visual inspection và hiệu năng runtime.

7.6. Đánh giá artifact

Artifact output cần được kiểm tra theo checklist:

- `manifest.json` tồn tại và khai báo đúng các artifact.
- `scene.sog` tồn tại và có kích thước khác 0.
- `occlusion.glb` tồn tại, load được bằng viewer hoặc validator, mesh không rỗng.
- `webar.zip` chứa `index.html`, PlayCanvas runtime, scene và mesh.
- Nếu `navmesh enabled`, `navmesh.glb` và `navmesh.bin` tồn tại khi manifest khai báo.
- Nếu `navmesh disabled` hoặc rỗng, output directory không còn file navmesh stale.
- Warning trong manifest được xem xét, không bỏ qua.

Checklist này phù hợp với mục tiêu sản phẩm: người dùng cuối không chỉ cần thuật toán chạy xong, mà cần bundle có thể chuyển giao và mở lại.

7.7. Đánh giá trực quan

Vì bài toán liên quan AR, kiểm tra trực quan vẫn cần thiết. Sau mỗi thay đổi lớn, nên render hoặc mở viewer để so sánh `scene.sog` với `occlusion.glb`. Các lỗi thường gặp gồm mesh bị xoay 90 độ, scale sai, occlusion mesh lệch khỏi scene, navmesh nằm trên tường thay vì sàn, hoặc artifact quá thô do voxel size lớn. Những lỗi này có thể không lộ qua unit test nếu fixture quá nhỏ.

Đánh giá trực quan nên kết hợp với metric. Nếu visual đúng nhưng triangle count quá cao, cần giảm mesh hoặc tăng voxel size. Nếu metric đẹp nhưng visual lệch, cần xem lại calibration. Nếu GPU nhanh nhưng mismatch cao so với CPU, cần điều tra shader hoặc precision.

7.8. Kế hoạch cập nhật số liệu

Khi bước thực nghiệm được tiến hành, báo cáo nên bổ sung một bảng kết quả có các cột: dataset, số splat, voxel size, backend, decode ms, voxel ms, mesh ms, export ms, collision triangles, GLB size, ZIP size, geometric RMS và warning. Nên có ít nhất một cảnh object, một cảnh interior và

một cảnh outdoor/terrain nếu dữ liệu có sẵn. Mỗi cảnh cần ghi rõ source, profile, reconstruction method và adapter command nếu dùng SuGaR/Poisson.

Bảng 7: Mẫu bảng kết quả sẽ điền sau khi benchmark được chốt.

Dataset	Splats	Backend	Voxel ms	Triangles	Ghi chú
Object	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	GPU	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	Kiểm tra occlusion prop.
Interior	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	GPU	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	Kiểm tra carve và navmesh.
Terrain	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	GPU	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	<i>chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại</i>	Kiểm tra floor-fill.

Việc để rõ *chưa chốt số liệu định lượng trong phạm vi báo cáo hiện tại* tốt hơn điền số không có căn cứ. Khi có output từ CLI benchmark, bảng này có thể cập nhật trực tiếp từ `summary.csv`.

8. Hạn chế và hướng phát triển

8.1. Hạn chế hiện tại

Hạn chế đầu tiên là calibration vẫn phụ thuộc vào người dùng. GUI đã đơn giản hóa workflow bằng hai scale endpoints và up axis, nhưng nếu người dùng chọn sai điểm hoặc sai trục, artifact sẽ sai. Hệ thống chưa có thuật toán tự động phát hiện mặt sàn đáng tin cậy từ toàn bộ point cloud hoặc Gaussian distribution. `floorNormal` trong recipe hiện được suy ra từ up axis, không phải kết quả plane fitting.

Hạn chế thứ hai là chất lượng mesh phụ thuộc mạnh vào voxel size, opacity threshold và profile. Một cấu hình phù hợp với object prop có thể không phù hợp với room-scale scene.

Exterior fill và carve giúp cảnh phòng nhưng có thể làm sai với object nhỏ. Floor fill hữu ích cho địa hình nhưng không đủ cho cảnh nhiều vật cản phức tạp. Do đó, profile hiện tại nên được xem là preset ban đầu, không phải giải pháp tự động tối ưu cho mọi cảnh.

Hạn chế thứ ba là adapter SuGaR/Poisson mới ở mức contract tích hợp. Hệ thống có selector, adapter command, request/response và validate mesh, nhưng chất lượng phụ thuộc vào adapter cụ thể và môi trường chạy ngoài. Để biến đây thành production feature, cần chuẩn hóa adapter, log lỗi chi tiết hơn, và có test với dataset thực.

Hạn chế thứ tư là benchmark định lượng chưa chốt. Codebase có lệnh benchmark và schema metric, nhưng báo cáo này chưa đưa số đo cuối. Điều đó làm phần đánh giá hiệu năng còn ở mức phương pháp. Tuy nhiên, đây là lựa chọn đúng hơn so với đưa số liệu chưa được xác minh.

8.2. Rủi ro kỹ thuật

Rủi ro kỹ thuật đáng chú ý nhất là GPU portability. `wgpu` giúp giảm phụ thuộc platform, nhưng desktop thực tế vẫn có khác biệt driver, adapter và quyền truy cập GPU. Nếu GPU path thất bại, người dùng cần cách chuyển sang CPU hoặc nhận thông báo rõ ràng. Vì GUI hiện mặc định GPU, phần fallback UX nên được cải thiện trong giai đoạn tiếp theo.

Rủi ro thứ hai là mesh semantic. Occlusion mesh và navmesh yêu cầu ý nghĩa hình học khác nhau, trong khi pipeline hiện chủ yếu dựa trên hình học tổng quát. Một bề mặt dựng tốt cho occlusion chưa chắc là mặt đi được. Ví dụ mặt bàn, ghế hoặc kệ có thể bị hiểu nhầm là vùng walkable nếu chỉ xét độ dốc. Để giải quyết, cần thêm semantic filtering, user annotation hoặc heuristic theo profile.

Rủi ro thứ ba là dữ liệu nguồn có nhiễu và floater. 3DGS từ ảnh thiếu hoặc camera path kém có thể sinh cụm Gaussian rời rạc, mờ hoặc nằm sai vị trí. Filter cluster và floater contribution có thể xử lý một phần, nhưng nếu bật sai có thể xóa chi tiết. Vì vậy workflow cần cơ chế preview trước/sau filter hoặc undo/redo ở GUI trong tương lai.

8.3. Hướng phát triển ngắn hạn

Hướng phát triển ngắn hạn gồm bốn nhóm. Nhóm thứ nhất là hoàn thiện GPU fallback: nếu GPU voxelization lỗi, GUI nên đề xuất chạy CPU hoặc tự retry theo cấu hình người dùng. Nhóm thứ hai là cải thiện profile UI: hiển thị ý nghĩa của object prop, interior room và outdoor terrain bằng preview ngắn, đồng thời cho phép advanced settings khi cần. Nhóm thứ ba là bổ sung benchmark chính thức với dataset đại diện. Nhóm thứ tư là tăng trực quan hóa output: overlay occlusion mesh và navmesh lên preview để người dùng phát hiện lỗi trước khi export ZIP.

8.4. Hướng phát triển dài hạn

Về dài hạn, hệ thống có thể mở rộng theo ba hướng. Hướng đầu tiên là automatic geometry understanding: nhận diện mặt sàn, tường, vật cản và vùng đi lại từ dữ liệu 3DGS hoặc từ dữ liệu phụ trợ. Hướng thứ hai là multi-resolution output: tạo nhiều mức chi tiết cho occlusion mesh, navmesh và render scene để phù hợp thiết bị AR khác nhau. Hướng thứ ba là tích hợp sâu hơn với runtime AR, ví dụ xuất metadata anchors, coordinate frame và occlusion material

theo chuẩn của engine đích.

Ngoài ra, adapter SuGaR/Poisson có thể được chuẩn hóa thành plugin chính thức. Mỗi plugin nên khai báo capability, input requirement, output schema, version và môi trường chạy. Khi đó GUI có thể phát hiện adapter, hiển thị trạng thái và tránh yêu cầu người dùng nhập command thủ công.

8.5. Định hướng đánh giá tiếp theo

Bước đánh giá tiếp theo cần có dataset thật từ các bối cảnh khác nhau: vật thể nhỏ, phòng trong nhà, hành lang hoặc địa hình ngoài trời. Mỗi dataset nên có ảnh preview, file nguồn, profile, config, artifact, manifest và nhận xét trực quan. Nên chạy mỗi cấu hình ít nhất ba lần để giảm nhiễu thời gian, ghi rõ phần cứng GPU/CPU, driver và hệ điều hành. Với AR, nên có thêm kiểm tra runtime: thời gian tải bundle, FPS viewer, độ đúng occlusion khi đặt vật thể ảo và khả năng navigation.

Khi có số liệu, phần thực nghiệm có thể bổ sung biểu đồ so sánh CPU/GPU, bảng dung lượng artifact và hình chụp overlay. Những bổ sung này sẽ làm báo cáo mạnh hơn mà không cần thay đổi cấu trúc chính.

9. Kết luận

Đề tài đã nghiên cứu và triển khai một công cụ xử lý dữ liệu 3D Gaussian Splatting phục vụ hệ thống AR. Trọng tâm của đề tài là chuyển dữ liệu 3DGS từ biểu diễn mạnh về hiển thị sang bộ artifact có cấu trúc hình học: scene render, occlusion mesh, manifest, WebAR bundle và navigation mesh tùy chọn. Hệ thống không thay thế renderer 3DGS, mà bổ sung lớp geometry cần thiết để vật thể ảo có thể tương tác hợp lý hơn với môi trường thật.

Về kiến trúc, hệ thống tách Rust core, CLI proof of concept và desktop GUI. Rust core đảm nhiệm các bước xử lý nặng và có kiểm thử độc lập. CLI hỗ trợ chạy batch và benchmark. Desktop app là giao diện cuối cùng, cho phép người dùng load .ply/.splat, preview scene, chỉnh position XYZ và rotation XYZ, đặt hai scale endpoints, chọn up axis, chọn reconstruction method và bake artifact. Thiết kế này cân bằng giữa khả năng tự động hóa và khả năng thao tác trực quan.

Về phương pháp, pipeline hiện có alignment bake, filter, voxelization CPU/GPU, fill/-carve, mesh extraction, reconstruction adapter SuGaR/Poisson, navmesh bằng rerecast và export WebAR. GPU voxelization được xem là đường chính trong GUI, trong khi CPU path vẫn giữ vai trò baseline và fallback cấu hình. SuGaR và Poisson được đặt trong kiến trúc adapter để hệ thống có thể mở rộng mà không buộc mọi dependency nghiên cứu vào core.

Về sản phẩm, đầu ra chính gồm manifest.json, scene.sog, occlusion.glb, webar.zip và navmesh tùy chọn. Manifest đóng vai trò hợp đồng dữ liệu và nguồn metric cho đánh giá. GUI hiển thị trạng thái, artifact và warning, giúp người dùng phát hiện lỗi bake rõ ràng hơn so với chỉ chạy command line.

Đề tài vẫn còn hạn chế ở calibration thủ công, chất lượng mesh phụ thuộc profile, adapter ngoài chưa được chuẩn hóa production và số liệu benchmark chưa chốt. Tuy nhiên, nền tảng hiện tại đã xác lập được pipeline, kiến trúc phần mềm và hợp đồng artifact cần thiết để tiếp tục phát triển. Các bước tiếp theo nên tập trung vào benchmark thực, overlay kiểm tra output trong

GUI, GPU fallback, adapter plugin và nhận diện hình học tự động cho mặt sàn/vật cản.

Tổng kết lại, công cụ đã tạo được cầu nối thực dụng giữa 3DGS và yêu cầu hình học của AR. Đây là hướng có giá trị nghiên cứu và ứng dụng vì nó giữ ưu điểm trực quan của 3DGS, đồng thời bổ sung dữ liệu mesh mà hệ thống AR cần để xử lý occlusion, collision và navigation.

Tài liệu tham khảo

- [1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4), 2023.
- [2] Antoine Guédon and Vincent Lepetit. SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Fidelity Rendering. *Proceedings of CVPR*, 2024.
- [3] William E. Lorensen and Harvey E. Cline. Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *Proceedings of SIGGRAPH*, 1987.
- [4] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson Surface Reconstruction. *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, 2006.
- [5] Mikko Mononen. Recast and Detour: Navigation Mesh Toolset for Games. <https://github.com/recastnavigation/recastnavigation>.
- [6] Jan Hohenheim. rerecast: Rust bindings and wrapper for Recast Navigation. <https://github.com/janhohenheim/rerecast>.
- [7] Tauri Team. Tauri Framework Documentation. <https://tauri.app>.
- [8] gfx-rs Community. wgpu: Safe and portable GPU programming in Rust. <https://github.com/gfx-rs/wgpu>.
- [9] PlayCanvas Team. PlayCanvas WebGL Engine Documentation. <https://playcanvas.com>.
- [10] Google. <model-viewer> Documentation. <https://modelviewer.dev>.